

AI Operations (AIOps)

Module 3: Infrastructure & Containerization

Course Content — 2 Lectures (6 Contact Hours) | Third-Year B.Sc./B.Tech. Data Science

Module Overview

Module 1 made a single model reproducible. Module 2 made data and compute scale across a cluster. Module 3 answers the question both of those modules quietly depend on: what actually IS “a machine” in a distributed AI system, and who decides which workload runs where? This module covers the infrastructure layer beneath every previous lecture — Docker for packaging reproducible, portable environments, Kubernetes for orchestrating those containers (and their GPUs) across a cluster, and KServe for turning a container into an autoscaling, production-grade inference endpoint.

Lecture	Title	Duration	Core Focus
L1	Docker & Containerization for AI	~3.0 hrs	Images vs. containers; Dockerfiles; multi-stage builds; GPU passthrough
L2	Kubernetes for AI/ML Workloads & Serverless Inference with KServe	~3.0 hrs	K8s architecture; GPU scheduling; Infrastructure as Code; KServe autoscaling

Module Learning Outcomes

- Explain why containerization is the natural infrastructure answer to Module 1's reproducibility protocol (CO2, CO9).
- Write a production-quality, multi-stage Dockerfile for an AI application, minimizing image size and build time (CO2).
- Configure GPU passthrough so a containerized workload can access host GPU resources (CO2, CO3).
- Describe Kubernetes' core objects (Pods, Deployments, Services, Jobs) and how they map onto AI workloads (CO2, CO3).
- Configure GPU resource requests/limits and scheduling constraints so a cluster is shared fairly and efficiently across teams (CO3).
- Apply Infrastructure as Code principles to manage reproducible, version-controlled cluster configuration (CO2, CO9).
- Deploy a trained model as an autoscaling, serverless inference endpoint using KServe, including a canary rollout (CO3, CO4).

Lecture 1 — Docker & Containerization for AI (≈3 hours)

1.1 Why Containers, After Everything We've Already Built?

Module 1 built a reproducibility protocol: Git for code, Conda/Mamba for the environment, DVC for data, MLflow for the run. That protocol has one weak link: a Conda environment file describes Python packages, but not the operating system, system libraries, CUDA/cuDNN versions, or anything outside the Python process itself. Two machines with an IDENTICAL environment.yml can still behave differently if their underlying OS or GPU drivers differ. Containers solve this by packaging the ENTIRE runtime environment — OS libraries included — into a single, portable unit.

Discussion prompt for class

Ask students: “You perfectly followed Module 1's protocol — pinned Git commit, pinned Conda environment, pinned DVC data. Your teammate still can't reproduce your result. What's left that ISN'T pinned?” Lead them toward: OS version, system libraries (e.g., libgomp, glibc), GPU driver/CUDA version, environment variables, anything installed outside pip/conda.

1.2 Images vs. Containers

- **Image** — a read-only, layered template: a filesystem snapshot plus metadata (what command to run, what ports to expose). Built once from a Dockerfile.
- **Container** — a running (or stopped) INSTANCE of an image, with its own writable layer on top. You can run many containers from one image.
- **Registry** — a place images are stored and shared (Docker Hub, a private registry, a cloud provider's container registry).

The image/container relationship is directly analogous to a class and an object in object-oriented programming: the image is the template, the container is the running instance.

1.3 Anatomy of a Dockerfile

A Dockerfile is a sequence of instructions, each producing one cached, reusable LAYER. Layers are stacked to form the final image.

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "train.py"]
```

Layer caching

Docker caches each layer and reuses it on the next build if nothing that layer depends on has changed. This is why requirements.txt is copied and installed BEFORE the rest of the application code: application code changes far more often than dependencies, so putting the expensive pip install step in an earlier, more stable layer means most rebuilds skip it entirely and reuse the cached layer.

1.4 Multi-Stage Builds

A naive Dockerfile for an AI application often ends up enormous — several GB — because build-time tools (compilers, dev headers, full CUDA toolkits) get baked into the final image alongside the actual runtime. Multi-stage builds solve this: use one stage to BUILD, and a second, minimal stage that only COPIES the finished artifacts across.

```
# --- Stage 1: build ---
FROM python:3.10 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/build/deps -r requirements.txt

# --- Stage 2: minimal runtime ---
FROM python:3.10-slim
WORKDIR /app
COPY --from=builder /build/deps /usr/local/lib/python3.10/site-packages
COPY . .
CMD ["python", "serve.py"]
```

The final image contains only the Python runtime and installed packages — no compilers, no build caches, no dev headers. This directly reduces image size, pull time, attack surface, and cold-start latency (a theme that returns in Lecture 2's KServe autoscaling discussion).

1.5 GPU Passthrough

By default, a container is isolated from the host's hardware, including GPUs. The NVIDIA Container Toolkit bridges this gap, letting a container see and use host GPUs as if they were its own.

```
# Build an image FROM an NVIDIA CUDA base image (matches host driver's CUDA version)
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04

RUN apt-get update && apt-get install -y python3 python3-pip
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python3", "train.py"]

# Run the container WITH GPU access (requires the NVIDIA Container Toolkit on the host)
$ docker run --gpus all my-training-image

# Restrict to a specific GPU (useful on a shared multi-GPU machine)
$ docker run --gpus '"device=0"' my-training-image
```

A common source of confusion: the CUDA version inside the container's base image must be COMPATIBLE with the host's installed NVIDIA driver, not identical to it — NVIDIA drivers are backward compatible with older CUDA runtime versions within a supported range, but a container demanding a newer CUDA version than the host driver supports will fail at the `docker run --gpus` step, not at build time.

1.6 Case Study: Optimizing GPU Access on a Shared University Lab Machine

Scenario: a single university lab workstation has 2 GPUs and is shared by several student research groups. Group A's container-based training job assumes it owns the entire machine and intermittently crashes or slows to a crawl when Group B is also training.

Guided in-class walkthrough

1. Diagnose: without explicit GPU restriction, both groups' containers pass `--gpus all`, so both processes contend for the SAME GPUs and their memory, causing out-of-memory errors and slowdowns for both.
2. Fix at the container level: assign each group's container an explicit device index (`--gpus '"device=0"'` vs. `"'device=1'"`), giving each group exclusive access to one GPU.
3. Discuss the limitation: this manual device-index assignment doesn't scale past a handful of users on one machine, and does nothing for FAIR scheduling over time (e.g., time-slicing a single GPU across many short jobs) — this motivates Lecture 2's cluster-level GPU scheduling with Kubernetes.

1.7 Hands-on Lab: Building a Multi-Stage, GPU-Ready Docker Image

Objective: containerize a small training script with a multi-stage Dockerfile, minimizing final image size, and configure it for GPU passthrough.

Step 1 — Write a naive, single-stage Dockerfile and check its size

```
$ docker build -t naive-image -f Dockerfile.naive .  
$ docker images naive-image # note the size
```

Step 2 — Rewrite as a multi-stage build and compare

```
$ docker build -t multistage-image -f Dockerfile.multistage .  
$ docker images multistage-image # compare size against naive-image
```

Step 3 — Add a .dockerignore to avoid copying unnecessary files

```
.git  
__pycache__  
*.pyc  
.venv  
data/  
*.ipynb_checkpoints
```

Step 4 — Run the container, with and without GPU access, and observe the difference

```
$ docker run multistage-image python3 -c "import torch; print(torch.cuda.is_available())"  
# Expect: False (no --gpus flag)  
  
$ docker run --gpus all multistage-image python3 -c "import torch; print(torch.cuda.is_available())"  
# Expect: True (if run on a GPU-equipped host with the NVIDIA Container Toolkit installed)
```

Lab target: every student produces a working multi-stage Dockerfile with a documented size reduction versus their naive first attempt, and can explain (even without physical GPU hardware in the classroom) exactly which flag and base image changes would be required to enable GPU passthrough.

Lecture 2 — Kubernetes for AI/ML Workloads & Serverless Inference with KServe (≈3 hours)

Part A: Kubernetes for AI/ML Workloads (≈100 minutes)

2.1 From One Container to a Fleet

Docker answers “how do I package one reproducible unit of work?” Kubernetes answers the next question: “I have hundreds of these containers and dozens of machines — who decides what runs where, restarts what fails, and shares resources fairly?” Kubernetes (K8s) is a container ORCHESTRATOR: it schedules containers onto a cluster of machines, restarts them on failure, and exposes them to the network.

2.2 Core Kubernetes Objects

Object	What It Is	AI/ML Relevance
Pod	The smallest deployable unit — one or more tightly-coupled containers sharing storage/network	One training or inference container usually = one Pod
Deployment	Manages a set of identical Pod replicas, handles rolling updates	Scaling an inference service horizontally
Job	Runs Pods to COMPLETION (not forever) — tracks success/failure, can retry	A training run, a batch scoring job
Service	A stable network endpoint routing traffic to a changing set of Pods	Exposing an inference endpoint at a fixed address
Node	A physical or virtual machine in the cluster that runs Pods	A GPU-equipped worker machine

2.3 Jobs vs. Deployments: Training vs. Serving

This distinction matters enormously for AI workloads specifically: a TRAINING run is naturally expressed as a Kubernetes Job (run to completion, potentially retried on failure, then done), while an INFERENCE service is naturally expressed as a Deployment (run forever, serving traffic, scaled up/down with demand). Using the wrong object type is a common early mistake — a training script deployed as a Deployment will be endlessly restarted by Kubernetes the moment it successfully finishes and exits.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: model-training-job
spec:
  backoffLimit: 3 # retry up to 3 times on failure
  template:
    spec:
      containers:
      - name: trainer
        image: my-registry/training-image:v3
      resources:
        limits:
```

```
nvidia.com/gpu: 1
restartPolicy: Never
```

2.4 GPU Scheduling & Resource Management

Kubernetes treats GPUs as a schedulable resource, similar to CPU and memory, via a device plugin (typically the NVIDIA device plugin) that advertises each node's GPU count to the scheduler.

```
resources:
  requests:
    nvidia.com/gpu: 1
  limits:
    nvidia.com/gpu: 1 # GPUs cannot be overcommitted -- request must equal limit
```

- Unlike CPU/memory, GPU requests and limits MUST be equal — Kubernetes does not support fractional or overcommitted GPU allocation out of the box.
- Node selectors and taints/tolerations let you steer GPU workloads specifically onto GPU-equipped nodes, and keep non-GPU workloads OFF expensive GPU nodes.
- This is the cluster-scale answer to Lecture 1's shared-lab-machine problem: instead of manually assigning device indices, the scheduler bin-packs GPU requests across all available nodes automatically and fairly.

2.5 Infrastructure as Code (IaC) Principles

Every Kubernetes object in this lecture is defined in a declarative YAML manifest, not created via ad hoc commands. This is Infrastructure as Code: cluster configuration is version-controlled, code-reviewed, and reproducible — the SAME reproducibility principle from Module 1, now applied to infrastructure itself rather than to a model.

- Declarative, not imperative: you describe the DESIRED end state ("3 replicas of this image"); Kubernetes continuously reconciles reality toward that state.
- Manifests live in Git, exactly like code and DVC pointers — a cluster's entire configuration history is auditable.
- Tools like Helm (templated manifests) and GitOps workflows (e.g., ArgoCD) build on this same principle at larger scale, automatically applying changes when a Git repository updates.

2.6 Case Study: A Fault-Tolerant Training Job That Survives Node Failures

Scenario: recall Module 2's Ray Train case study — a distributed training job across many GPU nodes needs to survive individual node failures without losing all progress.

Guided in-class walkthrough

1. A Kubernetes Job's backoffLimit lets the whole job retry if a Pod fails, but a naive retry restarts training FROM SCRATCH — wasteful for a multi-hour job.
2. Combine with Module 1's MLflow checkpointing: the training container periodically logs a checkpoint (to durable storage, not the container's ephemeral filesystem), so a retried Pod can resume from the last checkpoint instead of starting over.
3. Discuss persistent storage: a Pod's local filesystem disappears when the Pod is rescheduled — checkpoints MUST be written to a PersistentVolume or external object storage, never to the container's own disk, for this recovery pattern to work at all.

2.7 Case Study: Cost-Optimization of GPU Clusters Using Spot Instances

Recall Module 2's Ray discussion of spot/preemptible instances. At the Kubernetes layer, this is implemented with a dedicated spot-instance node pool, tainted so only workloads that explicitly tolerate interruption are scheduled there, combined with Pod priority classes so critical workloads preempt lower-priority ones if a spot node disappears unexpectedly.

Part B: Serverless AI Inference with KServe (~80 minutes)

2.8 The Inference Scaling Problem

An always-on Deployment running a large model, sized for peak traffic, wastes enormous GPU capacity during quiet periods (nights, weekends, low-traffic hours). KServe solves this with SERVERLESS inference: a model-serving Deployment that can scale down to ZERO replicas when idle, and scale up automatically — including from zero — the moment a request arrives.

2.9 The KServe InferenceService

KServe extends Kubernetes with a custom resource, the InferenceService, that wraps the low-level Deployment/Service/autoscaling machinery behind a single, simple, ML-specific declaration.

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: image-classifier
spec:
  predictor:
    model:
      modelFormat:
        name: mlflow
      storageUri: "models:/image-classifier/Production" # <- Module 1's Model Registry!
    resources:
      limits:
        nvidia.com/gpu: 1
```

Notice the direct connection back to Module 1: KServe's storageUri can point straight at an MLflow Model Registry stage or alias, so promoting a new model version in MLflow can trigger a new KServe rollout with zero manual repackaging.

2.10 Autoscaling, Including Scale-to-Zero

- KServe autoscales based on request CONCURRENCY (in-flight requests per replica), not raw CPU usage — more appropriate for inference workloads where request cost varies with input size.
- Scale-to-zero: with no traffic, KServe can terminate all replicas entirely, releasing expensive GPU resources. The first request after idling incurs a “cold start” — the cost that Lecture 1's minimal, multi-stage image directly reduces.
- This is why image size matters operationally, not just aesthetically: a smaller image pulls and starts faster, directly shortening cold-start latency for scale-to-zero services.

2.11 Canary Rollouts

KServe supports splitting traffic between two model versions by percentage, letting a new model version absorb a small fraction of live traffic before a full rollout — directly connecting to Module 1's Model Registry stages/aliases and previewing Module 4's deployment strategies (Canary, Blue-Green, Shadow).

```
spec:
  predictor:
    canaryTrafficPercent: 10 # 10% of traffic to the new revision, 90% to the current one
```

2.12 Case Study: Scaling an Image Classification Service During Peak Traffic

Scenario: a campus image-classification service (e.g., identifying plant species from photos for a botany course app) sees near-zero traffic overnight but a massive spike during a single 2-hour lab session when hundreds of students submit photos simultaneously.

Guided walkthrough

1. Overnight: KServe scales the service to zero replicas — no GPU cost incurred while idle.
2. At the start of the lab session: the first requests trigger a cold start; discuss the latency tradeoff students will observe (this is where Lecture 1's minimal image size pays off directly).
3. During the session: concurrency-based autoscaling adds replicas as request volume grows, up to a configured maximum.
4. After the session: traffic drops, KServe scales back down — eventually to zero again.
5. Discuss the alternative (an always-on Deployment sized for peak load) and its GPU-hours cost over a full week of mostly-idle time, to make the economic case for serverless inference concrete.

2.13 Hands-on Lab: Kubernetes Manifests & a KServe InferenceService

Objective: write and validate Kubernetes manifests for a training Job and a KServe InferenceService, including GPU resource requests and a canary rollout configuration.

Step 1 — Write and validate a training Job manifest

```
$ kubectl apply --dry-run=client -f training-job.yaml -o yaml
$ kubectl apply -f training-job.yaml
$ kubectl get jobs
$ kubectl logs job/model-training-job
```

Step 2 — Write and apply a KServe InferenceService manifest

```
$ kubectl apply -f inference-service.yaml
$ kubectl get inferencesservice image-classifier
$ kubectl get pods -l serving.kserve.io/inferencesservice=image-classifier
```

Step 3 — Send a test inference request

```
$ curl -H "Content-Type: application/json" \
  http://image-classifier.default.example.com/v1/models/image-classifier:predict \
  -d @sample_input.json
```

Step 4 — Configure and apply a canary rollout

```
$ kubectl patch inferencesservice image-classifier --type merge \
  -p '{"spec":{"predictor":{"canaryTrafficPercent": 10}}}'
$ kubectl get inferencesservice image-classifier -o yaml | grep -A2 traffic
```


Lab target: every student produces a validated training Job manifest and InferenceService manifest (YAML), and can explain how a change to `canaryTrafficPercent` would roll out safely in production, even where a live multi-node GPU cluster is not available in the classroom (a local single-node cluster such as `kind` or `minikube`, or dry-run validation, is used instead — see the companion notebook).

References for This Module

- Docker official documentation: docs.docker.com
- NVIDIA Container Toolkit documentation: docs.nvidia.com/datacenter/cloud-native/container-toolkit
- Kubernetes official documentation: kubernetes.io/docs
- KServe official documentation: kserve.github.io/website
- Kubernetes GPU scheduling guide: kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus